Second Year Engineering

Database Management Systems
(23CSE2022)

Class - S.Y. (Division-)

AY 2026-2027

SEM-I



Unit – III

Database Design



Syllabus

- Functional Dependency, Purpose of Normalization, Data Redundancy and Update Anomalies, Functional Dependency Single Valued Dependencies.
- Single Valued Normalization: 1NF, 2NF, 3NF, BCNF.
- Decomposition: lossless join decomposition and dependency preservation.
- Multi valued Normalization (4NF), Join Dependencies and the Fifth Normal Form.

Functional Dependency

- Functional dependency in DBMS, as the name suggests is a relationship between attributes of a table dependent on each other.
- Functional Dependency (FD) determines the relation of one attribute to another attribute in a database management system.
- Functional dependency helps you to maintain the quality of data in the database.
- A functional dependency is denoted by an arrow →.
- X → Y read as X determines Y
- Where X is the **determinant** attribute and Y is the **dependent** attribute.
- E.g. sid → sname

Types of Functional Dependency

1.Multivalued dependency

2.Trivial functional dependency

3.Non-trivial functional dependency

4.Transitive dependency

1. Multivalued dependency-Multivalued dependency occurs in the situation where there are multiple independent multivalued attributes in a single table.

2. Trivial functional dependency -The Trivial dependency is a set of attributes which are called a trivial if the set of attributes are included in that attribute.

So, X -> Y is a trivial functional dependency if Y is a subset of X.

3. Non-trivial functional dependency-

Functional dependency which also known as a nontrivial dependency occurs when A->B holds true where B is not a subset of A. In a relationship, if attribute B is not a subset of attribute A, then it is considered as a non-trivial dependency.

4. Transitive dependency-

A transitive is a type of functional dependency which happens when t is indirectly formed by two functional dependencies.

Properties of functional dependencies

1. Reflexivity: If Y is a subset of X then X □ Y and it is always valid.

   e.g. sid□sid

2. Augmentation: if X □ Y then XZ □ YZ

   e.g. sidphoneno □ snamephoneno

3. Transitive: if X □ Y and Y □ Z then X □ Z

   e.g. sid □ sname and sname □city then sid □city

4. Union: If X □ Y and X □ Z then X □Y Z

5. Decomposition: if X □Y Z then X □ Y and X □ Z

Purpose of Normalization

- If a database design is not perfect, it may contain anomalies, which are like a bad dream for any database administrator. Managing a database with anomalies is next to impossible.
- **Update anomalies** − If data items are scattered and are not linked to each other properly, then it could lead to strange situations.
- **Deletion anomalies** − We tried to delete a record, but parts of it was left undeleted because of unawareness, the data is also saved somewhere else.
- **Insert anomalies** − We tried to insert data in a record that does not exist at all.

Data Redundancy and Update Anomalies

A major aim of relational database design is to group attribute into relations to minimize data redundancy and thereby reduce the file storage space required by the implemented base relations.

Data redundancy is a condition created within a database or **data** storage technology in which the same piece of **data** is held in two separate places.

Codd's 12 Rule:

Rule 1: Information Rule

- The data stored in a database, may it be user data or metadata, must be a value of some table cell. Everything in a database must be stored in a table format.

Rule 2: Guaranteed Access Rule

- Every single data element (value) is guaranteed to be accessible logically with a combination of table-name, primary-key (row value), and attribute-name (column value). No other means, such as pointers, can be used to access data.

Rule 3: Systematic Treatment of NULL Values

- The NULL values in a database must be given a systematic and uniform treatment. This is a very important rule because a NULL can be interpreted as one the following − data is missing, data is not known, or data is not applicable.

Rule 4: Active Online Catalog

- The structure description of the entire database must be stored in an online catalog, known as **data dictionary**, which can be accessed by authorized users. Users can use the same query language to access the catalog which they use to access the database itself.

Rule 5: Comprehensive Data Sub-Language Rule

- A database can only be accessed using a language having linear syntax that supports data definition, data manipulation, and transaction management operations. This language can be used directly or by means of some application. If the database allows access to data without any help of this language, then it is considered as a violation.

Rule 6: View Updating Rule

- All the views of a database, which can theoretically be updated, must also be updatable by the system.

Rule 7: High-Level Insert, Update, and Delete Rule

- A database must support high-level insertion, updation, and deletion. This must not be limited to a single row, that is, it must also support union, intersection and minus operations to yield sets of data records.

Rule 8: Physical Data Independence

- The data stored in a database must be independent of the applications that access the database. Any change in the physical structure of a database must not have any impact on how the data is being accessed by external applications.

Rule 9: Logical Data Independence

- The logical data in a database must be independent of its user's view (application). Any change in logical data must not affect the applications using it. For example, if two tables are merged or one is split into two different tables, there should be no impact or change on the user application. This is one of the most difficult rule to apply.

Rule 10: Integrity Independence

- A database must be independent of the application that uses it. All its integrity constraints can be independently modified without the need of any change in the application. This rule makes a database independent of the front-end application and its interface.

Rule 11: Distribution Independence

- The end-user must not be able to see that the data is distributed over various locations. Users should always get the impression that the data is located at one site only. This rule has been regarded as the foundation of distributed database systems.

Rule 12: Non-Subversion Rule

- If a system has an interface that provides access to low-level records, then the interface must not be able to subvert the system and bypass security and integrity constraints.

Normalization

- Normalization is a database design technique that reduces data redundancy and eliminates undesirable characteristics like Insertion, Update and Deletion Anomalies.

Types of Normal forms

1. First Normal Form
2. Second Normal Form
3. Third Normal Form
4. BCNF (Boyce Codd Normal Form)
5. Fourth Normal Form
6. Fifth Normal Form

1st Normal Form (1NF)

- A table is referred to as being in its First Normal Form if atomicity of the table is 1.

- Here, atomicity states that a single cell cannot hold multiple values. It must hold only a single-valued attribute.

- The First normal form **disallows** the multi-valued attribute, composite attribute, and their combinations.

Let's understand the First Normal Form with the help of an example.

Below is a students' record table that has information about student roll number, student name, student course, and age of the student.

rollno	name	course	age
1	Rahul	c/c++	22
2	Harsh	java	18
3	Sahil	c/c++	23
4	Adam	c/c++	22
5	Lisa	java	24
6	James	c/c++	19
NULL	NULL	NULL	NULL

In the students record table, you can see that the course column has two values. Thus it does not follow the First Normal Form.

Now, if you use the First Normal Form to the previous table, you get the below table as a result.

rollno	name	course	age
1	Rahul	c	22
1	Rahul	c++	22
2	Harsh	java	18
3	Sahil	c	23
3	Sahil	c++	23
4	Adam	c	22
4	Adam	c++	22
5	Lisa	java	24
6	James	c	19
6	James	c++	19

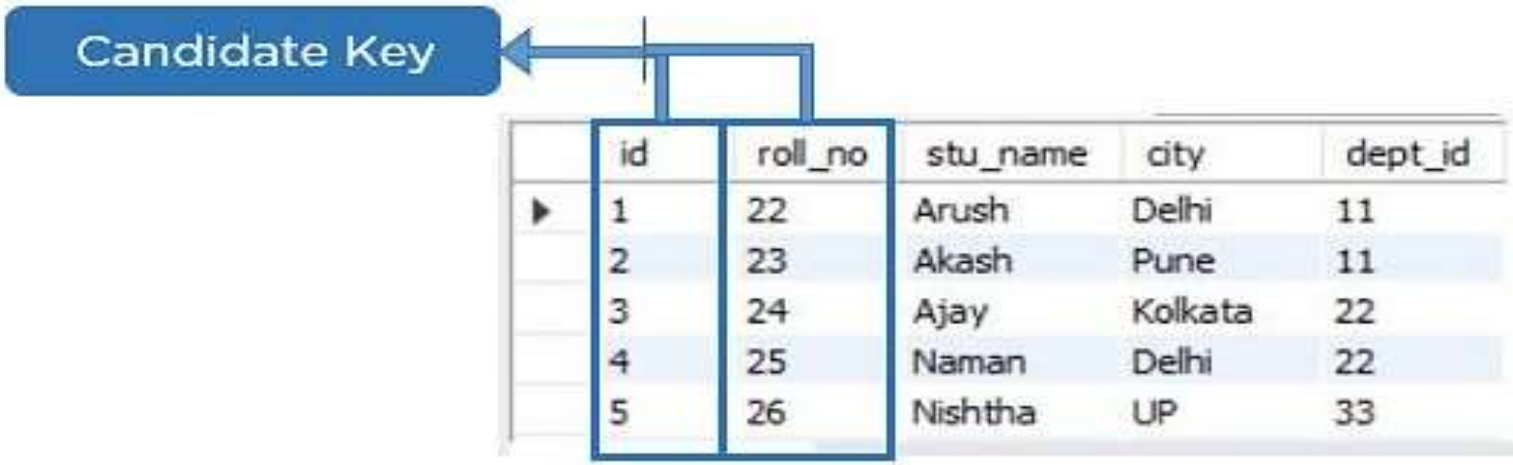
By applying the First Normal Form, you achieve atomicity, and also every column has unique values.

Before proceeding with the Second Normal Form, get familiar with Candidate Key and Super Key.

Candidate Key

A candidate key is a set of one or more columns that can identify a record uniquely in a table, and you can use each candidate key as a Primary Key.

Now, let's use an example to understand this better.



id	roll_no	stu_name	city	dept_id
1	22	Arush	Delhi	11
2	23	Akash	Pune	11
3	24	Ajay	Kolkata	22
4	25	Naman	Delhi	22
5	26	Nishtha	UP	33

Super Key

Super key is a set of over one key that can identify a record uniquely in a table, and the Primary Key is a subset of Super Key.
Let's understand this with the help of an example.

id	roll_no	stu_name	enroll_no	city	dept_id
1	22	Arush	223	Delhi	11
2	23	Akash	224	Pune	11
3	24	Ajay	225	Kolkata	22
4	25	Naman	226	Delhi	22
5	26	Nishtha	227	UP	33
6	27	Lamba	228	Haryana	44

Super Key

Example:

- Suppose we have the following Student table:

Roll_No	Student_Name	Email_ID	Subjects
101	Amit	amit@gmail.com	DBMS, Java, Python
102	Priya	priya@gmail.com	DBMS, Java
103	Rahul	rahul@gmail.com	Python, Java, DBMS

- Here, the **Subjects** column contains multiple values in a single cell. Therefore, this table **does not satisfy 1NF**.

- **Convert it into 1NF**
- Separate the multiple values into individual rows:

Roll_No	Student_Name	Email_ID	Subject
101	Amit	amit@gmail.com	DBMS
101	Amit	amit@gmail.com	Java
101	Amit	amit@gmail.com	Python
102	Priya	priya@gmail.com	DBMS
102	Priya	priya@gmail.com	Java
103	Rahul	rahul@gmail.com	Python
103	Rahul	rahul@gmail.com	Java
103	Rahul	rahul@gmail.com	DBMS

Second Normal Form (2NF)

- In the 2NF, relational must be in 1NF.
- In the second normal form, all non-key attributes are fully functional dependent on the primary key or candidate key.

Example: Let's assume, a school can store the data of teachers and the subjects they teach. In a school, a teacher can teach more than one subject.

- **TEACHER table**

TEACHER_ID	SUBJECT	TEACHER_AGE
25	Chemistry	30
25	Biology	30
47	English	35
83	Math	38
83	Computer	38

In the given table, non-prime attribute TEACHER_AGE is dependent on TEACHER_ID which is a proper subset of a candidate key. **That's why it violates the rule for 2NF.**
To convert the given table into 2NF, we decompose it into two tables:

TEACHER_DETAIL table:

TEACHER_ID	TEACHER_AGE
25	30
47	35
83	38

TEACHER_SUBJECT table:

TEACHER_ID	SUBJECT
25	Chemistry
25	Biology
47	English
83	Math
83	Computer



- **Example: Student Course Table**
- Suppose we have:

RollNo	CourseID	StudentName	CourseName	Marks
101	C01	Amit	DBMS	85
101	C02	Amit	Java	90
102	C01	Priya	DBMS	88
102	C02	Priya	Java	92

- Here, the primary key is **(RollNo, CourseID)**.
- But:

Studentname depends only on **RollNo**

Coursename depends only on **CourseID**

Marks depends on **RollNo + CourseID**

So Studentname and Coursename have **partial dependency**. Therefore, the table is **not in 2NF**.

- **Convert into 2NF**
- **Student Table**

RollNo	StudentName
101	Amit
102	Priya

- Course Table

CourseID	CourseName
C01	DBMS
C02	Java

- StudentCourse Table

RollNo	CourseID	Marks
101	C01	85
101	C02	90
102	C01	88
102	C02	92

Now:
- StudentName depends completely on RollNo
- CourseName depends completely on CourseID
- Marks depends on the complete key (RollNo, CourseID)

Therefore, the tables are in **2NF**.

Third Normal Form (3NF)

- A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.
- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.

A relation is in third normal form if it holds atleast one of the following conditions for every non-trivial function dependency $X \rightarrow Y$.

1. X is a super key.
2. Y is a prime attribute, i.e., each element of Y is part of some candidate key.

Example:

EMPLOYEE_DETAIL table:

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
222	Harry	201010	UP	Noida
333	Stephan	02228	US	Boston
444	Lan	60007	US	Chicago
555	Katharine	06389	UK	Norwich
666	John	462007	MP	Bhopal

Super key in the table above:

1. {EMP_ID}, {EMP_ID, EMP_NAME}, {EMP_ID, EMP_NAME, EMP_ZIP}....so on

Candidate key: {EMP_ID}

Non-prime attributes: In the given table, all attributes except EMP_ID are non-prime.

Here, EMP_STATE & EMP_CITY dependent on EMP_ZIP and EMP_ZIP dependent on EMP_ID. The non-prime attributes (EMP_STATE, EMP_CITY) transitively dependent on super key(EMP_ID). It violates the rule of third normal form.

That's why we need to move the EMP_CITY and EMP_STATE to the new <EMPLOYEE_ZIP> table, with EMP_ZIP as a Primary key.

EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_ZIP
222	Harry	201010
333	Stephan	02228
444	Lan	60007
555	Katharine	06389
666	John	462007

EMPLOYEE_ZIP table:

EMP_ZIP	EMP_STATE	EMP_CITY
201010	UP	Noida
02228	US	Boston
60007	US	Chicago
06389	UK	Norwich
462007	MP	Bhopal

- **Example: Student Table**
- Suppose we have

RollNo	StudentName	DeptID	DeptName
101	Amit	D01	Computer
102	Priya	D02	IT
103	Rahul	D01	Computer

Here, **RollNo** is the primary key.

Dependencies:

- RollNo → StudentName
- RollNo → DeptID
- DeptID → DeptName

So:

RollNo → DeptID → DeptName

DeptName depends on DeptID, not directly on the primary key Roll no.

This is called **transitive dependency**.

Therefore, the table is **not in 3NF**.

- **Convert into 3NF**
- **Student Table**

RollNo	StudentName	DeptID
101	Amit	D01
102	Priya	D02
103	Rahul	D01

- Department Table

DeptID	DeptName
D01	Computer
D02	IT

Now,

- RollNo → StudentName, DeptID
- DeptID → DeptName
- There is no non-key attribute depending on another non-key attribute in the **Student** table.

Boyce Codd Normal Form (BCNF)

Boyce Codd Normal Form is also known as 3.5 NF. It is the superior version of 3NF and was developed by Raymond F. Boyce and Edgar F. Codd to tackle certain types of anomalies which were not resolved with 3NF.

The first condition for the table to be in Boyce Codd Normal Form is that the table should be in the third normal form. Secondly, every Right-Hand Side (RHS) attribute of the functional dependencies should depend on the super key of that particular table.

For example :

You have a functional dependency X → Y. In the particular functional dependency, X has to be the part of the super key of the provided table.

Consider the subject table:

stuid	subject	professor
1	SQL	Prof. Mishra
2	Java	Prof. Anand
2	C++	Prof. Kanth
3	Java	Prof. James
4	DBMS	Prof. Lokesh

The subject table follows these conditions:

- Each student can enroll in multiple subjects.
- Multiple professors can teach a particular subject.
- For each subject, it assigns a professor to the student.

In the above table, student_id and subject together form the primary key because using student_id and subject; you can determine all the table columns.

Another important point to be noted here is that one professor teaches only one subject, but one subject may have two professors. Which exhibit there is a dependency between subject and professor, i.e. subject depends on the professor's name.

This table follows all the Normal forms except the **Boyce Codd Normal Form.**

As you can see stuid, and subject forms the primary key, which means the subject attribute is a prime attribute.

However, there exists yet another dependency - professor → subject.

BCNF does not follow in the table as a subject is a prime attribute, the professor is a non-prime attribute.

.

To transform the table into the BCNF, you will **divide the table into two parts.**

One table will hold stuid which already exists and the second table will hold a newly created column profid and in the second table will have the columns profid, subject, and professor, which satisfies the BCNF

	stuid	profid
▶	1	101
	2	102
	2	103
	3	102
	4	104

	profid	subject	professor
▶	1	SQL	Prof. Mishra
	2	Java	Prof. Anand
	2	C++	Prof. Kanth
	3	Java	Prof. James
	4	DBMS	Prof. Lokesh

COURSE_TEACHING

Course_ID	Instructor_ID	Room
C101	I101	R101
C102	I102	R102
C103	I103	R103

Suppose the following rules apply:

- Each course is taught by one instructor.
- Each instructor is assigned to only one course in this relation.

Then:

Course_ID → Instructor_ID

and

Instructor_ID → Course_ID

Both Course_ID and Instructor_ID can determine the other.

A violation of BCNF occurs when a determinant is not a candidate key.

BCNF Decomposition We can decompose the relation into:

COURSE_INSTRUCTOR

Course_ID	Instructor_ID
C101	I101
C102	I102
C103	I103

INSTRUCTOR_ROOM

Instructor_ID	Room
I101	R101
I102	R102
I103	R103

The decomposition ensures that every determinant is a candidate key/superkey in its respective relation.

Therefore: BCNF achieved.

Multivalued Dependency

•Multivalued dependency occurs when two attributes in a table are independent of each other but, both depend on a third attribute.

•A multivalued dependency consists of at least two attributes that are dependent on a third attribute that's why it always requires at least three attributes.

Example: Suppose there is a bike manufacturer company which produces two colors(white and black) of each model every year.

BIKE_MODEL	MANUF_YEAR	COLOR
M2011	2008	White
M2001	2008	Black
M3001	2013	White
M3001	2013	Black
M4006	2017	White
M4006	2017	Black

Here columns COLOR and MANUF_YEAR are dependent on BIKE_MODEL and independent of each other.

In this case, these two columns can be called as multivalued dependent on BIKE_MODEL.

The representation of these dependencies is shown below:

1. BIKE_MODEL → → MANUF_YEAR
2. BIKE_MODEL → → COLOR

This can be read as "BIKE_MODEL multidetermined MANUF_YEAR" and "BIKE_MODEL multidetermined COLOR".

Fourth normal form (4NF)

•A relation will be in 4NF if it is in Boyce Codd normal form and has no multi-valued dependency.

•For a dependency A → B, if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.

Example:

STUDENT

STU_ID	COURSE	HOBBY
21	Computer	Dancing
21	Math	Singing
34	Chemistry	Dancing
74	Biology	Cricket
59	Physics	Hockey

The given STUDENT table is in 3NF, but the COURSE and HOBBY are two independent entity. Hence, there is no relationship between COURSE and HOBBY.

In the STUDENT relation, a student with STU_ID, **21** contains two courses, **Computer** and **Math** and two hobbies, **Dancing** and **Singing**. So there is a Multi-valued dependency on STU_ID, which leads to unnecessary repetition of data.

So to make the above table into 4NF, we can decompose it into two tables:

STUDENT_HOBBY

STU_ID	HOBBY
21	Dancing
21	Singing
34	Dancing
74	Cricket
59	Hockey

STUDENT_COURSE

STU_ID	COURSE
21	Computer
21	Math
34	Chemistry
74	Biology
59	Physics

Relational Decomposition

•When a relation in the relational model is not in appropriate normal form then the decomposition of a relation is required.

•In a database, it breaks the table into multiple tables.

•If the relation has no proper decomposition, then it may lead to problems like loss of information.

•Decomposition is used to eliminate some of the problems of bad design like anomalies, inconsistencies, and redundancy.

Types of Decomposition

1. Lossless Decomposition
2. Dependency Preserving

Lossless Decomposition

- If the information is not lost from the relation that is decomposed, then the decomposition will be lossless.
- The lossless decomposition guarantees that the join of relations will result in the same relation as it was decomposed.
- The relation is said to be lossless decomposition if natural joins of all the decomposition give the original relation.

Example:
EMPLOYEE_DEPARTMENT table:

EMP_ID	EMP_NAME	EMP_AGE	EMP_CITY	DEPT_ID	DEPT_NAME
22	Denim	28	Mumbai	827	Sales
33	Alina	25	Delhi	438	Marketing
46	Stephan	30	Bangalore	869	Finance
52	Katherine	36	Mumbai	575	Production
60	Jack	40	Noida	678	Testing

The above relation is decomposed into two relations EMPLOYEE and DEPARTMENT

EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_AGE	EMP_CITY
22	Denim	28	Mumbai
33	Alina	25	Delhi
46	Stephan	30	Bangalore
52	Katherine	36	Mumbai
60	Jack	40	Noida

DEPARTMENT table

DEPT_ID	EMP_ID	DEPT_NAME
827	22	Sales
438	33	Marketing
869	46	Finance
575	52	Production
678	60	Testing

Now, when these two relations are joined on the common column "EMP_ID", then the resultant relation will look like:

Employee ⋈ Department

EMP_ID	EMP_NAME	EMP_AGE	EMP_CITY	DEPT_ID	DEPT_NAME
22	Denim	28	Mumbai	827	Sales
33	Alina	25	Delhi	438	Marketing
46	Stephan	30	Bangalore	869	Finance
52	Katherine	36	Mumbai	575	Production
60	Jack	40	Noida	678	Testing

Hence, the decomposition is Lossless join decomposition.

Dependency Preserving

- It is an important constraint of the database.
- In the dependency preservation, at least one decomposed table must satisfy every dependency.
- If a relation R is decomposed into relation R1 and R2, then the dependencies of R either must be a part of R1 or R2 or must be derivable from the combination of functional dependencies of R1 and R2.
- For example, suppose there is a relation R (A, B, C, D) with functional dependency set (A->BC). The relational R is decomposed into R1(ABC) and R2(AD) which is dependency preserving because FD A->BC is a part of relation R1(ABC).

Fifth normal form (5NF)

- A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.
- 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.
- 5NF is also known as Project-join normal form (PJ/NF).

Example

SUBJECT	LECTURER	SEMESTER
Computer	Anshika	Semester 1
Computer	John	Semester 1
Math	John	Semester 1
Math	Akash	Semester 2
Chemistry	Praveen	Semester 1

In the above table, John takes both Computer and Math class for Semester 1 but he doesn't take Math class for Semester 2.
In this case, combination of all these fields required to identify a valid data.

Suppose we add a new Semester as Semester 3 but do not know about the subject and who will be taking that subject so we leave Lecturer and Subject as NULL. But all three columns together acts as a primary key, so we can't leave other two columns blank.

So to make the above table into 5NF, we can decompose it into three relations P1, P2 & P3:

P1

SEMESTER	SUBJECT
Semester 1	Computer
Semester 1	Math
Semester 1	Chemistry
Semester 2	Math

P2

SUBJECT	LECTURER
Computer	Anshika
Computer	John
Math	John
Math	Akash
Chemistry	Praveen

P3

SEMSTER	LECTURER
Semester 1	Anshika
Semester 1	John
Semester 1	John
Semester 2	Akash
Semester 1	Praveen

Summary

Normal Form	Description
1NF	A relation is in 1NF if it contains an atomic value.
2NF	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.
3NF	A relation will be in 3NF if it is in 2NF and no transition dependency exists.
BCNF	A stronger definition of 3NF is known as Boyce Codd's normal form.
4NF	A relation will be in 4NF if it is in Boyce Codd's normal form and has no multi-valued dependency.
5NF	A relation is in 5NF. If it is in 4NF and does not contain any join dependency, joining should be lossless.